Mid-Term Assignment

Feature_Extraction_Techniques

Course Teacher: Prof. Dr. Md. Asraf Ali

Student Information

• Name: ABIR BOKHITAR

• ID: 22-47038-1

• Section: C [Fall 24-25]

What are Features?

Features are measurable properties or characteristics used to represent data in machine learning and data analysis. They help distinguish different classes or categories within the data.

Types of Features

- **Numerical**: Continuous (e.g., height, weight) or discrete values.
- **Categorical**: Categories or groups (e.g., color, type of animal).
- **Textual**: Derived from text (e.g., word counts, TF-IDF).
- **Image**: Characteristics from images (e.g., edges, textures).
- **Temporal**: Related to time (e.g., timestamps).

Importance

- **Data Representation**: Structured way to represent raw data.
- **Model Performance**: Quality of features impacts model accuracy.
- **Dimensionality Reduction**: Simplifies models while retaining essential information.

Feature Engineering

The process of selecting, modifying, or creating new features to improve model performance.

In summary, features are essential for building effective models and making predictions in data analysis and machine learning.

What is a feature in a raw image?

A feature in an image represents specific patterns, attributes, or characteristics that are unique or significant. These features can describe 2 types of patterns –

Low-level patterns - corners, textures, or colors.

High-level attributes - shapes, objects, or semantic content-like faces or scenes.

Features are used to -

- Identify important regions in the image (e.g., edges, corners).
- Describe the overall structure or patterns.
- Compare and recognize images for tasks like classification, object detection, or clustering.

Feature Extraction Techniques from Raw Image

Feature extraction is a crucial phase in image processing and computer vision that involves detecting and representing key structures within an image. These techniques are used to convert raw image data into numerical features that can be analyzed, while maintaining important details. These features play a significant role in various subsequent tasks, including object detection, classification, and image matching.

Description of some techniques

1. Histogram of Oriented Gradients (HOG)

HOG is a feature extraction technique used for detecting objects, particularly in tasks like pedestrian detection. It works by capturing edge structures and their orientations in an image.

Steps to Compute HOG Features:

1. **Preprocessing**:
   - Convert the image to grayscale, simplifying computations.
   - Optionally normalize the image to reduce the effect of lighting variations.
2. **Gradient Computation**:
   - Calculate gradients for each pixel along the x and y directions:
   $$G_x = \frac{\partial I}{\partial x}, \quad G_y = \frac{\partial I}{\partial y}$$
   - Compute the magnitude and orientation of gradients:
   $$|G| = \sqrt{G_x^2 + G_y^2}, \quad \theta = \tan^{-1}\left(\frac{G_y}{G_x}\right)$$
3. **Cell Division**:
   - Divide the image into small rectangular regions called cells (e.g., 8×8 pixels).
4. **Orientation Histograms**:
   - For each cell, create a histogram of gradient orientations, weighted by their magnitudes.
   - Commonly used bins: 99 orientations for 0∘ to 180∘
5. **Block Normalization**:
   - Group neighboring cells into blocks (e.g., 2×2 cells).
   - Normalize the histogram over the block to improve invariance to lighting.
6. **Feature Vector**:
   - Concatenate all normalized histograms to form the final feature vector.

Applications:

- Object detection, face recognition, and human pose estimation.

2. Scale-Invariant Feature Transform (SIFT)

SIFT is a robust technique for detecting and describing keypoints in images, invariant to scale, rotation, and affine transformations.

Steps to Compute SIFT Features:

1. **Keypoint Detection**:
    - Identify keypoints using a **difference of Gaussians (DoG)** at multiple scales.
    - The DoG helps detect regions of interest that vary significantly in intensity.
2. **Orientation Assignment**:
    - Assign an orientation to each keypoint based on the gradient magnitudes and orientations in its neighborhood.
    - This makes the keypoints rotation-invariant.
3. **Descriptor Generation**:
    - Divide the keypoint's neighborhood into smaller regions (e.g., 4×4 grid).
    - For each region, compute a histogram of gradient orientations (8 bins).
    - Concatenate these histograms into a 128-dimensional descriptor for each keypoint.
4. **Feature Matching**:
    - Match descriptors between images using techniques like Euclidean distance or ratio tests.

Applications:

- Object recognition, image stitching, and panorama generation.

3. Local Binary Patterns (LBP)

LBP is a simple yet powerful method for texture classification by encoding local intensity variations in an image.

Steps to Compute LBP Features:

1. **Grayscale Conversion**:
    - Convert the image to grayscale to simplify processing using, $I(x,y)=0.299 \cdot R(x,y)+0.587 \cdot G(x,y)+0.114 \cdot B(x,y)$
2. **Binary Pattern Computation**:

- For each pixel, compare its intensity with its neighbors.

$$b_i = \begin{cases} 1 & \text{if } I_{\text{neighbor},i} \geq I_{\text{center}} \\ 0 & \text{otherwise} \end{cases}$$

- Assign a binary code (1 if greater or equal, 0 otherwise):

$$LBP(x,y) = \sum_{i=0}^{P-1} b_i \cdot 2^i$$

3. **Uniform Patterns**:
   - Use the "uniform" method to capture patterns with at most two transitions (e.g., 0001110000011100).
   - This reduces dimensionality while retaining meaningful texture information.
4. **Histogram Calculation**:
   - Compute a histogram of LBP values for the image or region.
   - Normalize the histogram to form the final feature vector.

$$H_{\text{normalized}} = \frac{H}{\sum H}$$

Applications:

- Face recognition, texture analysis, and real-time object detection.

4. Convolutional Neural Networks (CNNs)

CNNs are deep learning models that automatically learn hierarchical features from images through a series of layers.

Steps to Extract Features Using CNNs:

1. **Input Layer**:
   - Accepts images in fixed dimensions (e.g., 224×224×3).
2. **Convolutional Layers**:
   - Apply convolutional filters (kernels) to detect features like edges, textures, or shapes.
   - Outputs **feature maps**.
3. **Activation Functions**:
   - Use non-linear activations like ReLU to introduce non-linearity.

4. **Pooling Layers**:
    - Perform downsampling (e.g., max pooling) to reduce the spatial dimensions while retaining essential features.
5. **Fully Connected Layers**:
    - Flatten feature maps into 1D vectors for final predictions or feature extraction.
6. **Pre-trained Models**:
    - Use models like VGG16, ResNet, or MobileNet to extract features from intermediate layers.

Applications:

- Classification, object detection, semantic segmentation, and clustering.

Code

Import Libraries

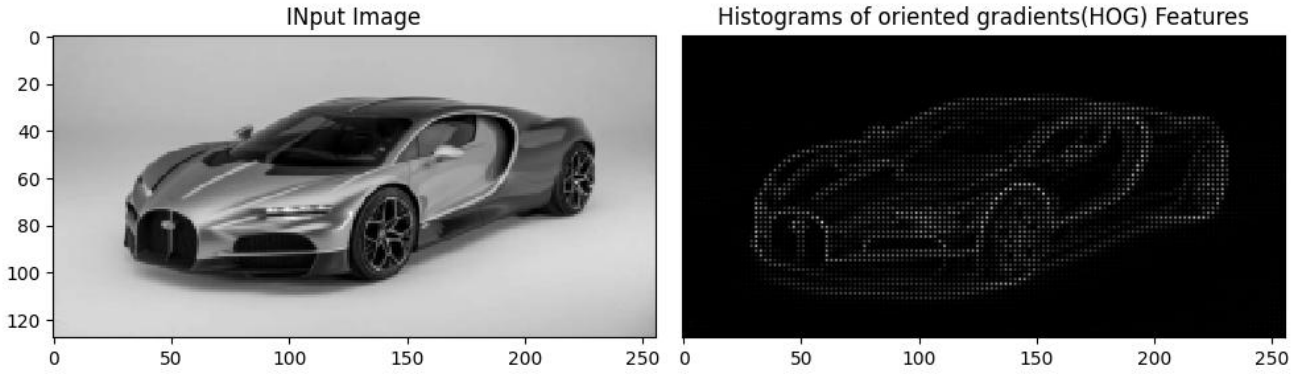
```
import numpy as np
from skimage.io import imread, imshow
from skimage.transform import resize
from skimage.feature import hog
from skimage import exposure
import matplotlib.pyplot as plt
import cv2
```

Image Preprocessing & HOG method

```
image = cv2.imread('Bugatti2.jpg', cv2.IMREAD_GRAYSCALE)
print(image.shape)

resized_img = resize(image, (128,256))

plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
plt.title('INput Image')
imshow(resized_img)
print(resized_img.shape)

fd, hog_image = hog(resized_img, orientations=9, pixels_per_cell=(2, 2),
                    cells_per_block=(2, 2), visualize=True)
plt.subplot(1,2,2)
plt.yticks([])
plt.imshow(hog_image, cmap='gray')
plt.title('Histograms of oriented gradients(HOG) Features')
plt.show()
```

Output-

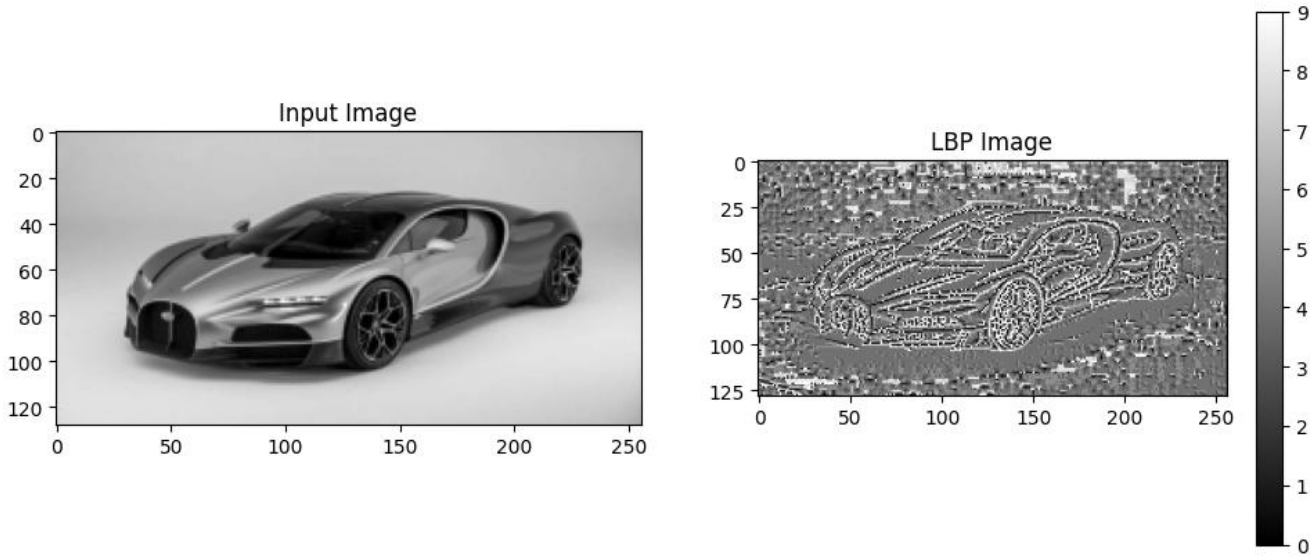
```
(349, 620)
(128, 256)
```



Local binary pattern (LBP) method

```
from skimage.feature import local_binary_pattern
lbp = local_binary_pattern(resized_img, P=8, R=1, method="uniform")

plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.imshow(resized_img, cmap="gray")
plt.title("Input Image")

plt.subplot(1,2,2)
plt.imshow(lbp, cmap="gray")
plt.title("LBP Image")
plt.colorbar()
plt.show()
```

Output-



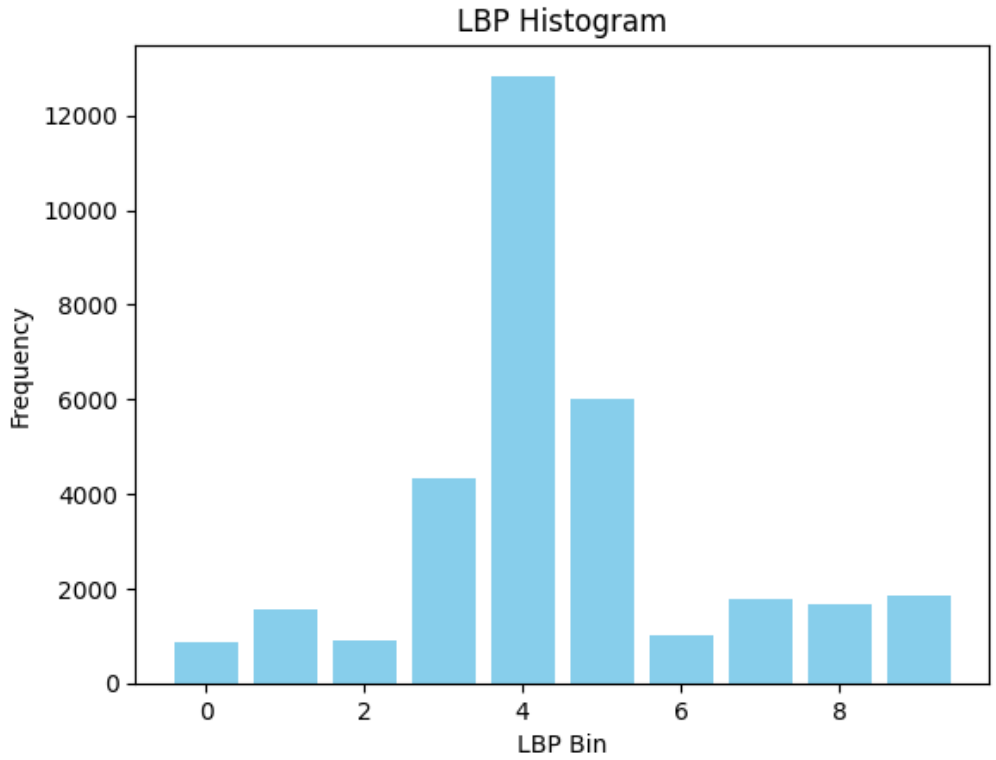
Plotting LBP using Histogram

In [6]:

```
hist_lbp, _ = np.histogram(lbp.ravel(), bins=np.arange(0, 11), range=(0, 10))
hist_lbp = hist_lbp.astype("float")
# hist_lbp /= hist_lbp.sum()

print("LBP Features:\n", hist_lbp)

plt.bar(range(len(hist_lbp)), hist_lbp, color="skyblue", width=0.8)
plt.title("LBP Histogram")
plt.xlabel("LBP Bin")
plt.ylabel("Frequency")
plt.show()
```

Output-

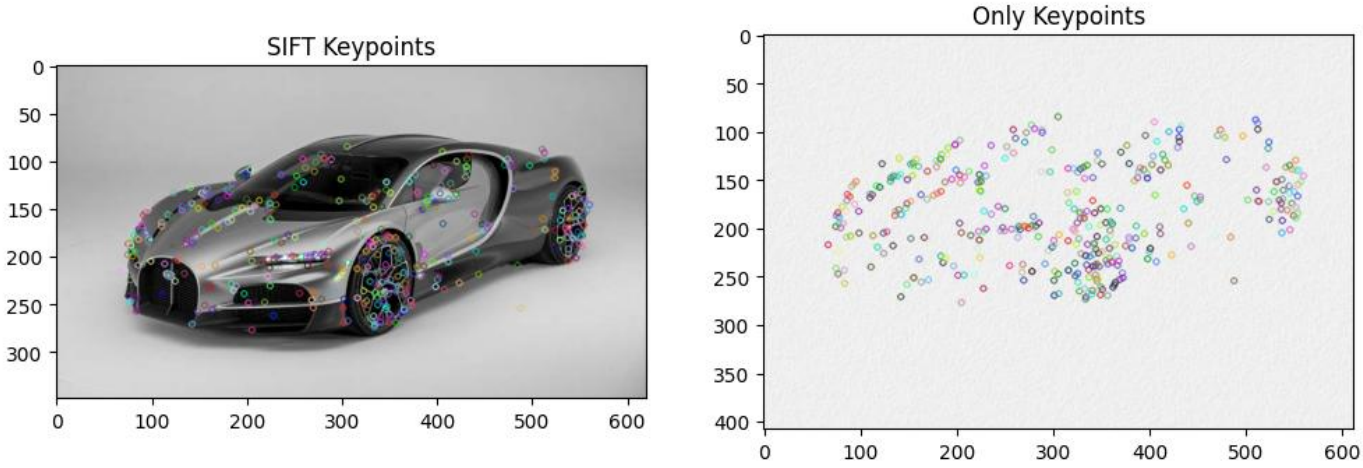
```
LBP Features:
 [  881.  1556.   887.  4313. 12837.  5993.  1001.  1773.  1679.  1848.]
```



Scale-Invariant Feature Transform (SIFT)

```
image = cv2.imread('Bugatti2.jpg', cv2.IMREAD_GRAYSCALE)
sift = cv2.SIFT_create()
keypoints, descriptors = sift.detectAndCompute(image, None)
keypoint_image = cv2.drawKeypoints(image, keypoints, None)

plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.imshow(keypoint_image, cmap='gray')
plt.title('SIFT Keypoints')

image2 = cv2.imread('blank.jpg', cv2.IMREAD_GRAYSCALE)
keypoint_blank = cv2.drawKeypoints(image2, keypoints, None)
plt.subplot(1,2,2)
plt.imshow(keypoint_blank, cmap='gray')
plt.title('Only Keypoints')
plt.show()
```

Output-



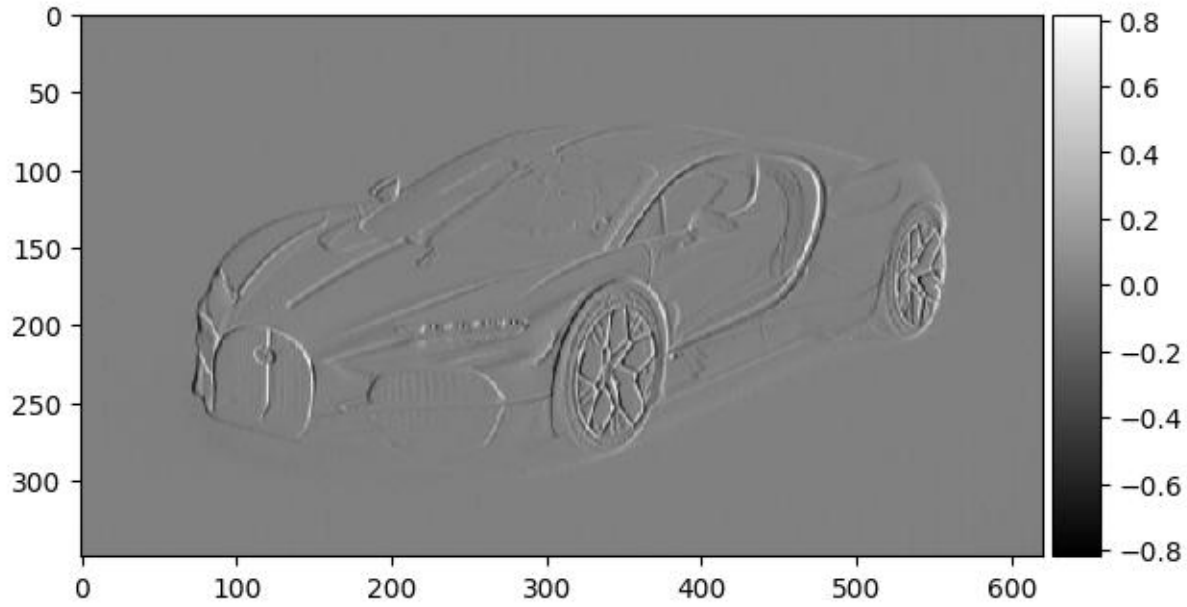
Prewitt edge detection

```
#importing the required libraries
import numpy as np
from skimage.io import imread, imshow
from skimage.filters import prewitt_h,prewitt_v
import matplotlib.pyplot as plt
```

```
# %matplotlib inline

image = imread('Bugatti2.jpg',as_gray=True)

#calculating horizontal edges using prewitt kernel
edges_prewitt_horizontal = prewitt_h(image)
#calculating vertical edges using prewitt kernel
edges_prewitt_vertical = prewitt_v(image)

imshow(edges_prewitt_vertical, cmap='gray')
```

Output-

```
<matplotlib.image.AxesImage at 0x7e4b52aa0130>
```



Pre-trained CNN

```
from tensorflow.keras.applications import VGG16
from tensorflow.keras.preprocessing.image import img_to_array, load_img
from tensorflow.keras.applications.vgg16 import preprocess_input
from tensorflow.keras.models import Model
import numpy as np

# Load the pre-trained VGG16 model without the top (classification) layer
model = VGG16(weights='imagenet', include_top=False)
```

```
# Load and preprocess the image
image_path = 'Bugatti2.jpg'
img = load_img(image_path, target_size=(224, 224))
img_array = img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)

# Extract features
features = model.predict(img_array)

# Print feature shape
print("Extracted Feature Shape:", features.shape)
```

Output-

```
1/1 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 762ms/step
Extracted Feature Shape: (1, 7, 7, 512)
```

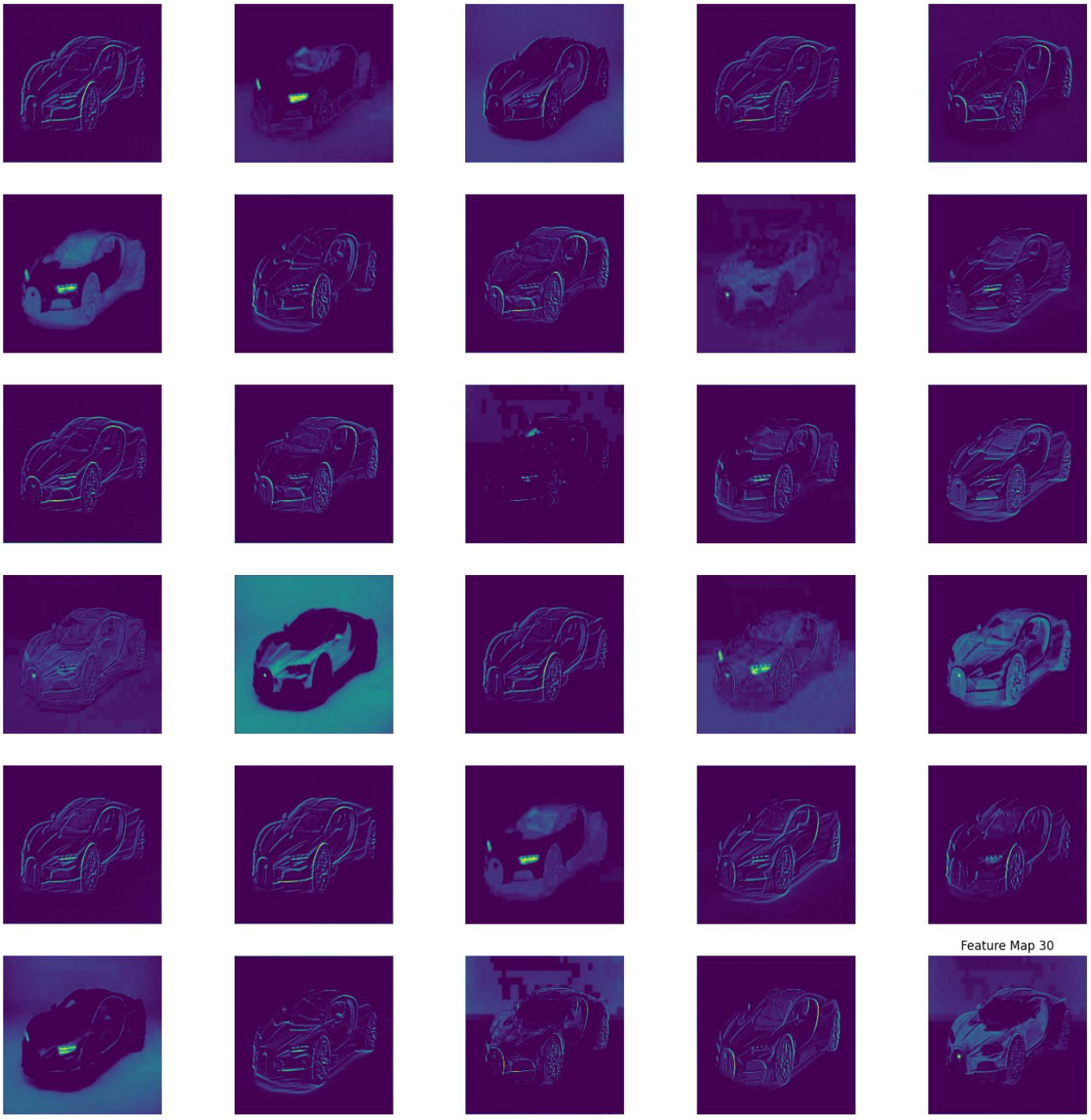
Feature mapping of each CNN layer output

In [15]:

```
import matplotlib.pyplot as plt

# Extract feature maps from a specific layer (e.g., the first conv layer)
intermediate_layer_model = Model(inputs=model.input,
outputs=model.get_layer('block1_conv1').output)
feature_maps = intermediate_layer_model.predict(img_array)

# Visualize the first 30 feature maps
plt.figure(figsize=(20, 20))
for i in range(30):
    plt.subplot(6, 5, i + 1)
    plt.imshow(feature_maps[0, :, :, i], cmap='viridis')
    plt.axis('off')
plt.title(f"Feature Map {i+1}")
plt.show()
```

Output-

```
1/1 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 50ms/step
```


Feature Map 30

Comparison of Techniques-

Technique	Feature Type	Applications	Key Strengths	Limitations
HOG	Edges, shapes	Object detection	Fast, robust to small deformations	Sensitive to large variations in scale
SIFT	Keypoints, descriptors	Image matching, object recognition	Invariant to scale and rotation	Computationally expensive
LBP	Texture	Face recognition, texture analysis	Efficient, robust to illumination changes	Limited discriminative power
CNN (Deep Learning)	Hierarchical patterns	Classification, clustering	Learns hierarchical features automatically	Requires large datasets and computation